

CS-317

Operating Systems

Welcome to Spring Semester 2020

3rd Year Computer Systems Engineering 2017

NED University of Engineering and Technology

About Instructor

- Name: Urooj Ainuddin
- Designation: Assistant Professor CIS NEDUET
- Office: Research Lab
- Office Extension: 2350
- Email: uroojain@neduet.edu.pk

Course Contents, Structure and Evaluation

The ways and means to help you understand what operating systems are...

Course Plan

Topic	Lectures
Introduction to Operating Systems	1
Processes	3
Uniprocessor Scheduling	2
Threads	4
Concurrency	8
I/O Management	2
File Management	3
Deadlocks	4
Networking and Distributed Processing	3
Total lectures	30

Books

01

Operating Systems:
Internals and Design
Principles, William
Stallings, 7th edition,
Prentice Hall, 2012

02

Operating System
Concepts, Abraham
Silberschatz, Peter Baer
Galvin and Greg Gagne,
9th edition, Wiley, 2013

03

Operating Systems,
Gary Nutt, 3rd edition,
Pearson, 2003.

Course Learning Outcomes

Sr. No.	CLOs	Taxonomy level	Programme learning outcome (PLO)	Assessment Tool
1.	Describe the fundamentals of operating system architecture	Cognitive C2	PLO-1 Engineering Knowledge	• Assignment • Final Exam
2.	Explore synchronization and resource management issues in operating systems	Cognitive C3	PLO-2 Problem Analysis	• Assignment • Final Exam

Why do we need this course?

- It extends your general knowledge regarding computers to a level required for a professional computer engineer.
- It helps you appreciate the ease that software provides you in harnessing the power of computer hardware.
- It brings together concepts of computer architecture and organization, and programming and algorithm development.

Let us talk about these times...

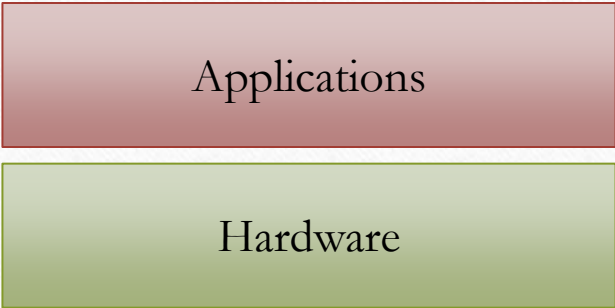
Online teaching is difficult. Online studying is difficult still. But...

Your willingness to learn will remove all
difficulties in our path.

Introduction to Operating Systems

Book 01 Chapter 02

What is an operating system?



Applications

The diagram consists of two stacked rectangular boxes. The top box is reddish-brown and contains the word 'Applications'. The bottom box is olive green and contains the word 'Hardware'. The boxes are centered horizontally and separated by a thin white gap.

Hardware

Pause the video!

Task# 1: Define operating system in your own words.

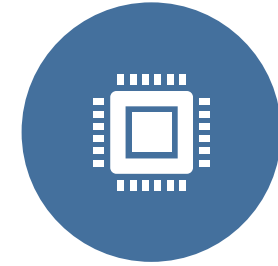
Write your answer in a word file. Save it as <my_roll_no>_lecture_1. If your roll no is 500, the file will be named 500_lecture_1. Do not add <> signs to the filename.

An operating system (OS) is system software that manages computer hardware and software resources and provides services to programs.

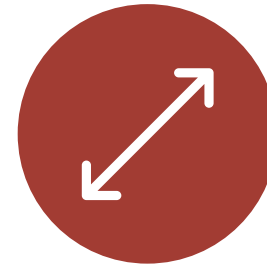
Objectives of an operating system



MAKE THE COMPUTER
CONVENIENT TO USE



ALLOW THE EFFICIENT
USE OF COMPUTER
RESOURCES



BE ABLE TO EVOLVE

Services provided by an operating system

1. Program Development

The OS provides tools and an environment to create programs.

Includes editors, compilers, debuggers

Helps programmers write, test, and modify code

The OS manages input/output devices.

Controls devices like keyboard, printer, disk

Provides a uniform interface to programs

Hides hardware complexity

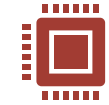


Program development



Program execution

The OS loads and runs programs.



Access to I/O devices



Access to files

The OS manages file systems.

Create, read, write, delete files

Organize data in directories

Ensures file security and access control



System access

The OS provides controlled access to system resources.

Uses system calls / APIs

Programs request services (memory, files, devices)

Prevents unauthorized access



Error detection and response



Accounting

The OS keeps track of resource usage.

Monitors CPU time, memory, disk usage

Useful for:

Billing (in multi-user systems)

Performance analysis

Helps in system optimization

Evolution of Operating Systems

The story of how it was and what led to how computers are today

Kinds of computers in history

- Serial processing systems
- Simple batch systems
- Multiprogrammed batch systems
- Time-sharing systems

Serial processing



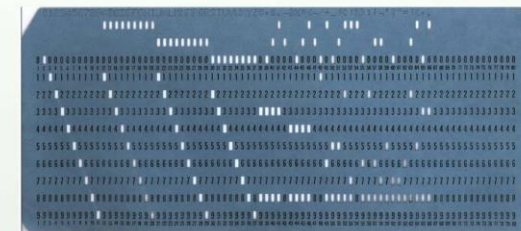
-
- Once upon a time there were computers, but they did not have any operating systems to manage them!
 - Computer consoles had display lights.
 - Computers had toggle switches and card readers for input.
 - Output was produced with a printer.
 - Programs were written in machine code on punched cards.
 - A stack of punched cards was fed into the card reader.

Have a look!



Punched Card
Reader

Punched card



<https://en.wikipedia.org/wiki/File:Blue-punch-card-front-horiz.png>

Problems with serial processing

Scheduling: Users needed to go to the computer facility, pay a certain amount of money, and get a time slot reserved for their computation. If the reserved time slot was larger than required, the computer remained idle in the remainder of the slot. If the time slot was smaller than required, the user would be asked to remove his program input from the computer at the expiry of the slot. He would have to reserve a larger time slot in which he would start from the beginning. This would mean that whatever he did in this reservation was wasted, as was the money that he paid for it.

Problems with serial processing

Setup Time: Users had to

- Load the compiler,
- Load the high-level language source code, ... this would generate the object code,
- Save and load the object file,
- Load the library functions and link them,

all of which involved mounting and dismounting punched cards. If there was an error, the user would have to start from the beginning.

In a nutshell...



-
- Computers were scarce. computer resources are limited and not available in abundance.
 - Computer time was expensive to buy.
 - Too much time was spent in setup.
 - An error could drastically affect the cost of computation.

Simple batch systems

- So we made a program called a **monitor**, which was meant to manage the computer for us.
- The user no longer had direct access to the processor.
- The user submitted his job, the source code, on punched cards. A computer operator took jobs from different users, placed them one after the other in a batch, and mounted the batch on the card reader, for the monitor.
- Execution began with the monitor, which read the first job, and allowed it to run on the processor. The monitor **transferred control** to the job. When the job finished, **control would automatically be transferred** to the monitor, which read the next job and transferred control to it. This continued until the batch of jobs was complete.

Simple batch systems

- Now the memory had to store two programs at the same time, the monitor and the user program. Memory locations that contained monitor code formed the **system address space**. Memory locations that contained the user program formed the **user address space**.
- The monitor had to be loaded as soon as the machine was started and would remain in memory as long as the machine was turned on. It could never be moved out of memory, or the machine would not function. So it was called the **resident monitor**.
- mid 2022 The monitor could write at any memory location, but the user program could not write in the system address space. This feature, called **memory protection**, ensured that a malicious program did not change the monitor, causing the computer to malfunction.
- To stop a user job from monopolizing the system, a **timer** was set at the beginning of a user job. If the timer expired before the job had ended, control was forcibly transferred from the job to the monitor.

Format of a user job submitted to a simple batch system

- Lines starting with the \$ sign are lines of the Job Control Language (JCL). The JCL lines are meant to be executed by the monitor. They instruct the monitor to prepare the machine for a user job.
- Any line not beginning with a \$ sign is part of the code of the user job.

\$JOB

\$FTN

•
•
•



FORTRAN Instructions

\$LOAD

\$RUN

•
•
•



Data

\$END

Simple batch systems

- Some instructions which altered critical system registers and handled I/O operations were designated as **privileged instructions**.
- Two modes of operation were designated for the processor, **kernel mode** and **user mode**. Privileged instructions could only be executed in kernel mode.
- When control transferred to the monitor, it would switch the processor's mode to kernel, and switch it back to user mode before transferring control to the user job. This mode switch could only be done by the monitor. This guaranteed that the privileged instructions could never be executed by the user job.

In a nutshell...

- The processor either worked on a monitor instruction or a user job instruction at a time.
- All I/O was handled by the monitor only.
- If a user job tried accessing system address space or executing privileged instruction, control was immediately transferred back to the monitor.
- Later computers also incorporated the use of interrupts.

Multiprogrammed batch systems

- **Multiprogramming:** Having more than one user jobs loaded for execution in the memory of a computer. It is also called **multitasking**.
- **Point to note:** If there is a single processor, it will execute only one instruction, and hence only one program at a time.
- **Degree of multiprogramming** = The number of user jobs loaded in the memory.
- We started loading more than one user jobs in the user address space of the computer.

General structure of a job



- Every user job is composed of CPU bursts and I/O bursts. In a CPU burst, the job executes on the processor. In an I/O burst, an I/O operation that the job requested takes place.
- The red chunks are I/O bursts. The yellow chunks are CPU bursts.
- Every job starts and ends with a CPU burst.
- Every I/O burst is sandwiched between two CPU bursts.
- Jobs are either CPU intensive or I/O intensive. CPU intensive jobs have long CPU bursts and short I/O bursts. I/O intensive jobs have short CPU bursts and long I/O bursts.
- When a job requests I/O, the job waits for the operation to finish. During this time, the processor is idle.

Multiprogrammed batch systems

- Every user job is loaded in a finite set of memory locations, which forms its address space. No user job can write in the system address space. No user job can write in another job's address space.
- When the current job asks for I/O, the monitor selects another user job to execute on the processor. This reduces processor idle time and makes the machine more efficient.
- A job leaves the processor if it has completed or if it wants to have an I/O operation performed.
- When an I/O device completes the designated task, it interrupts the monitor. When the current job relinquishes the processor, the monitor identifies the user job that had requested the task, and transfers control to it.

interrupts introduced

Pause the video!

Task# 2: What happens when the degree of multiprogramming is very low?

Write your answer in a word file previously saved as <my_roll_no>_lecture_1.

with low degree of multiprog the idle time of proccessor will be large in case no other job to be placed in pplace of i/o blocked job

Time-sharing systems

- Multiprogrammed batch systems were mostly mainframes, i.e. they were computers which would support multiple users simultaneously, each with his own input and output device. There was a single processor. Jobs from each user were kept in the memory. At one instant, only one job would execute. All other users experienced a lag in service and had to wait for their turn.
- Time-sharing is a technique created to bluff mainframe users into believing that each of them had a separate processor working on his job only.
- We set up a timer for a period called the **time slot** or the **time quantum**. We start the timer at the beginning of the user job. When the timer expires, we take the processor away from the current job. If it has not finished executing, we place it at the end of the queue of jobs submitted. We pick a job from the head of this queue, start the timer, and allow it to execute on the processor.

Time-sharing systems

- When the timer expires, control is automatically transferred to the monitor and the current job is said to be **preempted**. **Preemption** means taking the processor forcibly away from a user job. The monitor then transfers control to the job at the head of the queue.
- The size of the quantum is small. This results in very fast interleaving of execution of multiple user jobs, which generates constant activity at all user terminals attached to the mainframe. This makes each user believe the processor is working on his job.
- When a job asks for I/O, control is transferred back to the monitor, which starts the I/O operation, resets the timer, and passes control to the job at the head of the queue.

Kinds of computers in history

- Serial processing systems
- Simple batch systems
- Multiprogrammed batch systems
- Time-sharing systems

What did we learn?

- A machine cannot be run efficiently without an operating system (OS).
- The OS must not be altered by user programs.
- All I/O must be managed by the OS.
- Keeping more than one user jobs in memory reduces the processor's idle time.
- Interleaving jobs and switching between them at a fast rate improves user experience and reduces response time.

The systems today...

- All computers have well-developed operating systems.
- All systems use multiprogramming and time-sharing.

Things to do

- Read the book.
- I will ask you to submit the word file previously saved as <my_roll_no>_lecture_1 in Google classroom.
- Write your questions on a piece of paper and keep it in front of you during the online session.
- If you have suggestions relevant to content or quality of this lecture, email them to me.